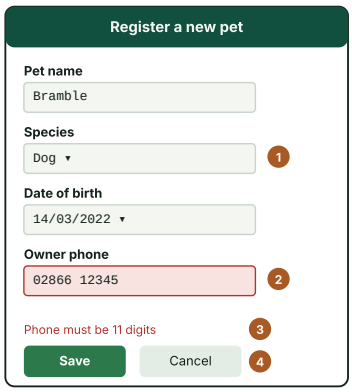
# System design: storyboarding, prototyping and data design

AP06 FACT FILE

## Specification: students should be able to

- Describe the purpose of systems design, development and testing and the contents of a design document
- Describe the use of storyboarding in developing the user interface
- Describe the use of prototyping in the design of the user interface and the data model
- Describe the design of inputs, outputs and user interfaces, applying principles of human–computer interaction (HCI)
- Describe the design of data structures and the use of a data dictionary
- Explain the role of validation and verification in design and describe common validation checks

**Key words:** design document, storyboarding, navigation, prototyping, look and feel, stub, data model, human–computer interaction (HCI), input design, output design, data dictionary, data structure, validation, verification, range check, format check, consistency check




**HCI features**

1 Drop-down list prevents typing errors
2 Field highlighted where the error is
3 Clear, helpful error message (feedback)
4 Buttons in the same place on every screen

**Data dictionary (extract)**

| Field | Type | Validation |
|---|---|---|
| PetName | String(30) | Presence |
| Species | String(10) | Lookup list |
| DateOfBirth | Date | Range: not future |
| OwnerPhone | String(11) | Length, format |

An annotated input screen design for registering a pet, with numbered HCI features, beside part of its data dictionary

## Overview: from requirements to a design

Systems design is the creative stage that follows analysis. Because the designer has to understand exactly what the system is for, design can only begin once the user requirements have been agreed, and the finished design must meet every one of those requirements. The result is a design document. It sets out the user interface and navigation between screens, the inputs and how they will be captured, the outputs and reports, the data structures and data dictionary, the validation and verification each data item needs, algorithms for the main processes and an initial test plan. Development follows: programmers build the system from this document, usually as a set of separate modules that are tested on their own and again as they are joined together. Two techniques are used heavily during design to involve users before money is spent on coding. Storyboarding sketches each screen and the routes between them, and prototyping builds an early working model that users can try. Both aim to find problems while they are still cheap to fix.

## Storyboarding

Storyboarding comes from the film industry, where a story was drawn as a sequence of sketches so that the crew could picture each scene and spot problems before filming. In system design each frame of the storyboard represents a screen or page. The designer records what each screen contains, identifies every input and output on it, and draws arrows to show navigation from one screen to another. The storyboard is then talked through with users, who can say where the route is too long, where a feature is missing or where a screen is confusing. Its strengths are that it is quick and cheap to produce, needs no technical knowledge to understand, encourages communication between designers and users early in the project and allows changes to be made before any code exists. Its weaknesses are that it is not interactive, shows only limited detail and cannot convey how fast or responsive the finished system will feel, so it cannot fully judge usability. Storyboards are therefore often followed by a prototype.

### Prototyping the interface and data model

A prototype is an early, deliberately incomplete version of the software that gives users the look and feel of the proposed system. It may contain stubs, which are menu options or buttons that appear but do nothing yet, and it often has little error checking, so it may only work when given perfectly correct data. For the user interface, a prototype lets users compare menu arrangements, colour schemes and screen layouts, and lets the client and developer confirm that the system will meet the agreed requirements. For the data model, building a prototype forces the designer to decide what data must be input, what outputs are needed and what processing links them, which reveals the tables and relationships the database requires. Prototyping improves the quality of the requirements, finds problems and enhancements earlier and reduces misunderstandings because users can react to something real. However, it can be costly and time-consuming, users may be swayed by appearance rather than accuracy or security, an early version may be accepted too readily, and users may expect the finished system far sooner than is realistic.

### HCI, input and output design

Input design decides what data is captured, how it is captured and how screens or forms are laid out. Good input design reduces typing and errors by using drop-down lists, option buttons, date pickers and sensible default values. Output design decides the content, layout and medium of screens, printed reports and documents, matched to each audience: a manager may need a summary chart, while a clerk needs a detailed list. Human–computer interaction principles guide these decisions. Consistency means the same layout, colours and wording on every screen, so users learn the system quickly. Feedback means confirming actions and giving clear error messages that say how to put the problem right. Error prevention restricts input and asks for confirmation before destructive actions such as deleting. A low memory load means showing choices rather than expecting users to remember codes. Accessibility covers contrast, scalable text, keyboard navigation and labels for screen readers, and avoids using colour alone to carry meaning. Finally, the interface should suit its users, offering shortcuts for experts and guidance for novices.

### Data structures and the data dictionary

Designers must decide how data will be held: as database tables with keys and relationships, as files of records, or as arrays and lists inside a program. These decisions are recorded in a data dictionary, a central description of every data item in the system. For each field it typically gives the name, data type, length or size, format, a description, an example value and the validation rule, and it notes primary and foreign keys and the relationships between tables. The data dictionary keeps a development team consistent, because every programmer uses the same names, types and rules; without it, one module might store a date as text and another as a date value, causing faults when modules are joined. It also gives testers the exact limits to test and helps maintenance staff understand the data years later. A validation rule in a data dictionary should be precise, for example a range check of 1.00 to 50.00 pounds, rather than simply naming the type of check.

### Validation and verification

Validation is an automatic check carried out by the software to make sure data is sensible and obeys set rules before it is accepted. A presence check ensures a field is not left blank; a type check ensures, for example, that a number has been entered; a range check tests that a value lies between limits; a length check limits the number of characters; a format check matches a pattern such as a postcode; a lookup check compares the entry with a list of allowed values; a check digit detects errors in codes such as barcodes; and a consistency or cross-field check compares fields, such as making sure a return date is after a loan date. Validation cannot prove that data is correct: a pupil whose balance is really 12.60 pounds might be entered as 16.20, which is still within range. Verification checks that data has been entered or transferred accurately, for instance by double entry of a password or email address, or by proof-reading the screen against the original document. Both are specified during design.

## T Key terms from the mark schemes

**Design document**
The output of the design phase, setting out the user interface, inputs, outputs and reports, data structures, validation and verification, algorithms and an initial test plan for the new system.
`Credited: design specification`

**Storyboarding**
A design technique in which each screen of a proposed interface is sketched, showing its contents, inputs and outputs and the navigation between screens, to obtain early feedback from users.

**Prototyping**
Building an early, incomplete working version of a system so that users can experience its look and feel and give feedback on the interface and data requirements.
Credited: prototype

**Data dictionary**
A central record of every data item in a system, giving its name, data type, length, format, description, example and validation rules.

**Verification**
A check that data has been entered or transferred accurately, such as double entry or proof-reading against the source.

**Consistency check**
A validation check that compares two or more fields to make sure their values are compatible, e.g. return date after loan date.
Credited: cross-field check

**Stub**
A part of a prototype or program, such as a menu option or subroutine, that is present but has no working functionality yet.
Credited: placeholder

**Validation**
An automatic check by the system that data is reasonable and follows set rules before it is accepted; it cannot ensure the data is correct.

**Human–computer interaction (HCI)**
The study and design of how people interact with computer systems so that interfaces are usable, efficient and accessible.

**Design specification**
The deliverable of the design stage describing the inputs, outputs, interfaces, data structures, processes, security and test plan of the new system.
Credited: design document

## C C# example

Prototyping the interface and the data model

```
// Prototype main menu: only option 1 is built
Console.WriteLine("1. View today's menu");
Console.WriteLine("2. Order history");
Console.WriteLine("3. Top up balance");
string choice = Console.ReadLine();
if (choice == "1")
    Console.WriteLine("Chicken curry £2.40 | Pasta bake £2.10");
else
    Console.WriteLine("Coming soon"); // stub
```

### Exam focus

Short questions often ask you to list contents of a design document; give specific items such as interface design, data structures, validation and verification required, report design and a test plan. For storyboarding, describe what a storyboard shows (contents of each screen, inputs and outputs, navigation) and why it helps (early user feedback, no specialist knowledge needed). Prototyping questions are often in two parts: explain its use for the interface and for the data model separately, then give advantages and disadvantages. Each point usually earns one mark, so two advantages and two disadvantages make four marks, and they must be different ideas, not the same point reversed. For validation, name the check and state the exact rule for that field, for example a range check that the age is between 11 and 19 inclusive; a bare name such as range check may only earn half the credit. Many candidates confuse validation with verification, so learn one clear example of each. In design questions, justify each feature by saying how it helps the user, linking it to an HCI principle.

## ? Check yourself

1. A Craigavon software company is designing a new booking system for a driving school. State four items that should be included in its design document. [4]
2. Describe how storyboarding could be used when designing the screens of a mobile app for booking pitches at a GAA club. [3]
3. Give two advantages and two disadvantages of using storyboarding to design a user interface. [4]

**Answers:** 1. Any four from: design of the user interface/screens and navigation (1) / input design (1) / output and report design (1) / specification of data structures/data dictionary (1) / validation and verification required (1) / algorithms for processes (1) / security or access levels (1) / a test plan (1)  2. Each screen of the app (e.g. login, choose pitch, choose time, confirm) is sketched as a frame showing its contents (1) / the inputs and outputs on each screen are identified, e.g. date selected, available slots shown (1) / arrows show navigation between screens, and the storyboard is walked through with club members to get feedback/spot awkward routes before coding (1)  3. Advantages, two from: quick/cheap to create (1) / promotes early communication between designer and users (1) / no specialist knowledge needed to understand (1) / early feedback on functionality and navigation (1) / changes can be made early/cheaply (1). Disadvantages, two from: not interactive (1) / limited detail (1) / cannot show speed, responsiveness or usability accurately (1)